

JUnit, Mockito and SLF4J - Hands-on Solutions

1. JUnit Basic Testing Exercises

Exercise 1: Setting Up JUnit

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>
```

Added this to pom.xml, then created a new class CalculatorTest under src/test/java.

Exercise 2: Writing Basic JUnit Tests

```
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }
}
```

```
import org.junit.Test;
import static org.junit.Assert.*;

public class CalculatorTest {

    Calculator calc = new Calculator();

    @Test
    public void testAdd() {
        assertEquals(5, calc.add(2, 3));
    }

    @Test
    public void testSubtract() {
        assertEquals(1, calc.subtract(3, 2));
    }
}
```

Exercise 3: Assertions in JUnit

```
import org.junit.Test;
import static org.junit.Assert.*;

public class AssertionsTest {

    @Test
    public void testAssertions() {
        assertEquals(5, 2 + 3);
        assertTrue(5 > 3);
    }
}
```

```

        assertFalse(5 < 3);
        assertNull(null);
        assertNotNull(new Object());
    }
}

```

Exercise 4: AAA Pattern, Setup and Teardown

```

import org.junit.Before;
import org.junit.After;
import org.junit.Test;
import static org.junit.Assert.*;

public class CalculatorAAATest {

    Calculator calc;

    @Before
    public void setUp() {
        calc = new Calculator();
        System.out.println("Setting up before test");
    }

    @After
    public void tearDown() {
        calc = null;
        System.out.println("Cleaning up after test");
    }

    @Test
    public void testAddUsingAAA() {
        // Arrange
        int a = 4, b = 6;

        // Act
        int result = calc.add(a, b);

        // Assert
        assertEquals(10, result);
    }
}

```

2. Advanced JUnit Testing Exercises

Exercise 1: Parameterized Tests

```

public class EvenChecker {
    public boolean isEven(int number) {
        return number % 2 == 0;
    }
}

```

```

import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.ValueSource;
import static org.junit.jupiter.api.Assertions.*;

public class EvenCheckerTest {

    EvenChecker checker = new EvenChecker();

    @ParameterizedTest
    @ValueSource(ints = {2, 4, 6, 8, 10})
    void testIsEvenTrue(int number) {

```

```

    assertTrue(checker.isEven(number));
}

@ParameterizedTest
@ValueSource(ints = {1, 3, 5, 7})
void testIsEvenFalse(int number) {
    assertFalse(checker.isEven(number));
}
}

```

Exercise 2: Test Suites and Categories

```

import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;

@Suite
@SelectClasses({ CalculatorTest.class, EvenCheckerTest.class })
public class AllTests {
    // kept empty on purpose, this class is just a holder for the suite annotations
}

```

Needed to add the junit-platform-suite dependency in pom.xml for @Suite to work.

Exercise 3: Test Execution Order

```

import org.junit.jupiter.api.*;

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
public class OrderedTests {

    @Test
    @Order(1)
    void firstTest() {
        System.out.println("Running first test");
    }

    @Test
    @Order(2)
    void secondTest() {
        System.out.println("Running second test");
    }

    @Test
    @Order(3)
    void thirdTest() {
        System.out.println("Running third test");
    }
}

```

Exercise 4: Exception Testing

```

public class ExceptionThrower {
    public void throwException() {
        throw new IllegalArgumentException("Invalid argument passed");
    }
}

```

```

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class ExceptionThrowerTest {

```

```

@Test
void testExceptionIsThrown() {
    ExceptionThrower thrower = new ExceptionThrower();

    IllegalArgumentException exception = assertThrows(
        IllegalArgumentException.class,
        () -> thrower.throwException()
    );

    assertEquals("Invalid argument passed", exception.getMessage());
}
}

```

Exercise 5: Timeout and Performance Testing

```

public class PerformanceTester {
    public void performTask() throws InterruptedException {
        Thread.sleep(200); // simulating some work being done
    }
}

```

```

import org.junit.jupiter.api.Test;
import java.time.Duration;
import static org.junit.jupiter.api.Assertions.*;

public class PerformanceTesterTest {

    @Test
    void testTaskCompletesInTime() {
        PerformanceTester tester = new PerformanceTester();
        assertTimeout(Duration.ofMillis(500), () -> tester.performTask());
    }
}

```

3. Mockito Hands-On Exercises

Support classes used across these exercises:

```

public interface ExternalApi {
    String getData();
    String getDataById(int id);
}

public class MyService {
    private ExternalApi api;

    public MyService(ExternalApi api) {
        this.api = api;
    }

    public String fetchData() {
        return api.getData();
    }

    public String fetchDataById(int id) {
        return api.getDataById(id);
    }
}

```

Exercise 1: Mocking and Stubbing

```
import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class MyServiceTest {

    @Test
    public void testExternalApi() {
        ExternalApi mockApi = mock(ExternalApi.class);
        when(mockApi.getData()).thenReturn("Mock Data");

        MyService service = new MyService(mockApi);
        String result = service.fetchData();

        assertEquals("Mock Data", result);
    }
}
```

Exercise 2: Verifying Interactions

```
import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;

public class VerifyInteractionTest {

    @Test
    public void testVerifyInteraction() {
        ExternalApi mockApi = mock(ExternalApi.class);
        MyService service = new MyService(mockApi);

        service.fetchData();

        verify(mockApi).getData();
    }
}
```

Exercise 3: Argument Matching

```
import static org.mockito.Mockito.*;
import static org.mockito.ArgumentMatchers.*;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class ArgumentMatchingTest {

    @Test
    public void testArgumentMatching() {
        ExternalApi mockApi = mock(ExternalApi.class);
        when(mockApi.getDataById(anyInt())).thenReturn("Some Data");

        MyService service = new MyService(mockApi);
        String result = service.fetchDataById(5);

        assertEquals("Some Data", result);
        verify(mockApi).getDataById(eq(5));
    }
}
```

Exercise 4: Handling Void Methods

```
public interface Logger {
    void log(String message);
}
```

```

public class LoggerService {
    private Logger logger;

    public LoggerService(Logger logger) {
        this.logger = logger;
    }

    public void process() {
        logger.log("Processing started");
    }
}

```

```

import static org.mockito.Mockito.*;
import static org.mockito.ArgumentMatchers.*;
import org.junit.jupiter.api.Test;

public class LoggerServiceTest {

    @Test
    public void testVoidMethodIsCalled() {
        Logger mockLogger = mock(Logger.class);
        doNothing().when(mockLogger).log(anyString());

        LoggerService service = new LoggerService(mockLogger);
        service.process();

        verify(mockLogger).log("Processing started");
    }
}

```

Exercise 5: Mocking and Stubbing with Multiple Returns

```

import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class MultiReturnTest {

    @Test
    public void testMultipleReturns() {
        ExternalApi mockApi = mock(ExternalApi.class);
        when(mockApi.getData())
            .thenReturn("First Call")
            .thenReturn("Second Call");

        MyService service = new MyService(mockApi);

        assertEquals("First Call", service.fetchData());
        assertEquals("Second Call", service.fetchData());
    }
}

```

Exercise 6: Verifying Interaction Order

```

import static org.mockito.Mockito.*;
import org.mockito.InOrder;
import org.junit.jupiter.api.Test;

public class OrderVerificationTest {

    @Test
    public void testInteractionOrder() {
        ExternalApi mockApi = mock(ExternalApi.class);

```

```

mockApi.getData();
mockApi.getDataById(1);

InOrder inOrder = inOrder(mockApi);
inOrder.verify(mockApi).getData();
inOrder.verify(mockApi).getDataById(1);
}
}

```

Exercise 7: Handling Void Methods with Exceptions

```

import static org.mockito.Mockito.*;
import static org.mockito.ArgumentMatchers.*;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class VoidExceptionTest {

    @Test
    public void testVoidMethodThrowsException() {
        Logger mockLogger = mock(Logger.class);
        doThrow(new RuntimeException("Logging failed"))
            .when(mockLogger).log(anyString());

        assertThrows(RuntimeException.class, () -> mockLogger.log("test message"));
    }
}

```

4. Advanced Mockito Hands-On Exercises

Exercise 1: Mocking Databases and Repositories

```

public interface Repository {
    String getData();
}

public class Service {
    private Repository repository;

    public Service(Repository repository) {
        this.repository = repository;
    }

    public String processData() {
        return "Processed " + repository.getData();
    }
}

```

```

import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class ServiceTest {

    @Test
    public void testServiceWithMockRepository() {
        Repository mockRepository = mock(Repository.class);
        when(mockRepository.getData()).thenReturn("Mock Data");

        Service service = new Service(mockRepository);
        String result = service.processData();

        assertEquals("Processed Mock Data", result);
    }
}

```

```
}  
}
```

Exercise 2: Mocking External Services (RESTful APIs)

```
public interface RestClient {  
    String getResponse();  
}  
  
public class ApiService {  
    private RestClient restClient;  
  
    public ApiService(RestClient restClient) {  
        this.restClient = restClient;  
    }  
  
    public String fetchData() {  
        return "Fetched " + restClient.getResponse();  
    }  
}
```

```
import static org.mockito.Mockito.*;  
import org.junit.jupiter.api.Test;  
import static org.junit.jupiter.api.Assertions.*;  
  
public class ApiServiceTest {  
  
    @Test  
    public void testServiceWithMockRestClient() {  
        RestClient mockRestClient = mock(RestClient.class);  
        when(mockRestClient.getResponse()).thenReturn("Mock Response");  
  
        ApiService apiService = new ApiService(mockRestClient);  
        String result = apiService.fetchData();  
  
        assertEquals("Fetched Mock Response", result);  
    }  
}
```

Exercise 3: Mocking File I/O

```
public interface FileReader {  
    String read();  
}  
  
public interface FileWriter {  
    void write(String content);  
}  
  
public class FileService {  
    private FileReader reader;  
    private FileWriter writer;  
  
    public FileService(FileReader reader, FileWriter writer) {  
        this.reader = reader;  
        this.writer = writer;  
    }  
  
    public String processFile() {  
        String content = reader.read();  
        writer.write(content);  
        return "Processed " + content;  
    }  
}
```



```

import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class FileServiceTest {

    @Test
    public void testServiceWithMockFileIO() {
        FileReader mockFileReader = mock(FileReader.class);
        FileWriter mockFileWriter = mock(FileWriter.class);
        when(mockFileReader.read()).thenReturn("Mock File Content");

        FileService fileService = new FileService(mockFileReader, mockFileWriter);
        String result = fileService.processFile();

        assertEquals("Processed Mock File Content", result);
    }
}

```

Exercise 4: Mocking Network Interactions

```

public interface NetworkClient {
    String connect();
}

public class NetworkService {
    private NetworkClient client;

    public NetworkService(NetworkClient client) {
        this.client = client;
    }

    public String connectToServer() {
        return "Connected to " + client.connect();
    }
}

```

```

import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class NetworkServiceTest {

    @Test
    public void testServiceWithMockNetworkClient() {
        NetworkClient mockNetworkClient = mock(NetworkClient.class);
        when(mockNetworkClient.connect()).thenReturn("Mock Connection");

        NetworkService networkService = new NetworkService(mockNetworkClient);
        String result = networkService.connectToServer();

        assertEquals("Connected to Mock Connection", result);
    }
}

```

Exercise 5: Mocking Multiple Return Values

```

import static org.mockito.Mockito.*;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class MultiReturnServiceTest {

```

```

@Test
public void testServiceWithMultipleReturnValues() {
    Repository mockRepository = mock(Repository.class);
    when(mockRepository.getData())
        .thenReturn("First Mock Data")
        .thenReturn("Second Mock Data");

    Service service = new Service(mockRepository);

    String firstResult = service.processData();
    String secondResult = service.processData();

    assertEquals("Processed First Mock Data", firstResult);
    assertEquals("Processed Second Mock Data", secondResult);
}
}

```

5. Spring Testing Exercises

Support classes used across these exercises:

```

@Entity
public class User {
    @Id
    private Long id;
    private String name;
    // getters and setters
}

public interface UserRepository extends JpaRepository<User, Long> {
    List<User> findByName(String name);
}

@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;

    public User getUserById(Long id) {
        return userRepository.findById(id).orElse(null);
    }

    public User saveUser(User user) {
        return userRepository.save(user);
    }
}

@Service
public class CalculatorService {
    public int add(int a, int b) {
        return a + b;
    }
}

@RestController
@RequestMapping("/users")
public class UserController {
    @Autowired
    private UserService userService;

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        return ResponseEntity.ok(userService.getUserById(id));
    }
}

```

```

    @PostMapping
    public ResponseEntity<User> createUser(@RequestBody User user) {
        return ResponseEntity.ok(userService.saveUser(user));
    }
}

@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(NoSuchElementException.class)
    public ResponseEntity<String> handleNotFound(NoSuchElementException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body("User not found");
    }
}

```

Exercise 1: Basic Unit Test for a Service Method

```

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CalculatorServiceTest {

    CalculatorService service = new CalculatorService();

    @Test
    void testAdd() {
        assertEquals(8, service.add(3, 5));
    }
}

```

Exercise 2: Mocking a Repository in a Service Test

```

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import java.util.Optional;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
public class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private UserService userService;

    @Test
    void testGetUserById() {
        User user = new User();
        user.setId(1L);
        user.setName("Sneha");

        when(userRepository.findById(1L)).thenReturn(Optional.of(user));

        User result = userService.getUserById(1L);

        assertEquals("Sneha", result.getName());
    }
}

```

Exercise 3: Testing a REST Controller with MockMvc

```
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.Mockito.*;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(UserController.class)
public class UserControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;

    @Test
    void testGetUser() throws Exception {
        User user = new User();
        user.setId(1L);
        user.setName("Sneha");

        when(userService.getUserById(1L)).thenReturn(user);

        mockMvc.perform(get("/users/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.name").value("Sneha"));
    }
}
```

Exercise 4: Integration Test with Spring Boot

```
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.context.SpringBootTest.WebEnvironment;
import org.springframework.boot.test.web.client.TestRestTemplate;
import org.springframework.http.ResponseEntity;

import static org.junit.jupiter.api.Assertions.*;

@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
public class UserIntegrationTest {

    @Autowired
    private TestRestTemplate restTemplate;

    @Test
    void testFullFlow() {
        ResponseEntity<User> response = restTemplate.getForEntity("/users/1", User.class);
        assertEquals(200, response.getStatusCodeValue());
    }
}
```

Exercise 5: Test Controller POST Endpoint

```
@Test
void testCreateUser() throws Exception {
    User newUser = new User();
    newUser.setName("Ravi");
}
```

```

when(userService.saveUser(any(User.class))).thenReturn(new User);

mockMvc.perform(post("/users")
    .contentType(MediaType.APPLICATION_JSON)
    .content("{\"name\":\"Ravi\"}"))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.name").value("Ravi"));
}

```

Added this test method inside the same `UserControllerTest` class from Exercise 3.

Exercise 6: Test Service Exception Handling

```

@Test
void testGetUserByIdNotFound() {
    when(userRepository.findById(99L)).thenReturn(Optional.empty());

    User result = userService.getUserById(99L);

    assertNull(result);
}

```

Added this to the `UserServiceTest` class from Exercise 2.

Exercise 7: Test Custom Repository Query

```

import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.beans.factory.annotation.Autowired;
import org.junit.jupiter.api.Test;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;

@DataJpaTest
public class UserRepositoryTest {

    @Autowired
    private UserRepository userRepository;

    @Test
    void testFindByName() {
        User user = new User();
        user.setName("Meena");
        userRepository.save(user);

        List<User> results = userRepository.findByName("Meena");

        assertEquals(1, results.size());
        assertEquals("Meena", results.get(0).getName());
    }
}

```

Exercise 8: Test Controller Exception Handling

```

@Test
void testUserNotFoundHandledGlobally() throws Exception {
    when(userService.getUserById(100L)).thenThrow(new NoSuchElementException());

    mockMvc.perform(get("/users/100"))
        .andExpect(status().isNotFound())
}

```

```

        .andExpect(content().string("User not found"));
    }

```

Added this to the `UserControllerTest` class as well, after adding `@ExceptionHandler(GlobalExceptionHandler.class)` is picked up automatically by `@WebMvcTest`.

Exercise 9: Parameterized Test with JUnit

```

import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import static org.junit.jupiter.api.Assertions.*;

public class CalculatorParamTest {

    CalculatorService service = new CalculatorService();

    @ParameterizedTest
    @CsvSource({
        "2,3,5",
        "10,5,15",
        "0,0,0",
        "-1,1,0"
    })
    void testAddWithMultipleInputs(int a, int b, int expected) {
        assertEquals(expected, service.add(a, b));
    }
}

```

6. Mocking Dependencies in Spring Tests using Mockito

Exercise 1: Mocking a Service Dependency in a Controller Test

```

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.http.ResponseEntity;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
public class UserControllerUnitTest {

    @Mock
    private UserService userService;

    @InjectMocks
    private UserController userController;

    @Test
    void testGetUser() {
        User user = new User();
        user.setId(1L);
        user.setName("Kiran");

        when(userService.getUserById(1L)).thenReturn(user);

        ResponseEntity<User> response = userController.getUser(1L);

        assertEquals("Kiran", response.getBody().getName());
    }
}

```

```
}  
}
```

Exercise 2: Mocking a Repository in a Service Test

Same approach as Module 5, Exercise 2 - using `@Mock` for the repository and `@InjectMocks` for the service.

```
@ExtendWith(MockitoExtension.class)  
public class UserServiceMockTest {  
  
    @Mock  
    private UserRepository userRepository;  
  
    @InjectMocks  
    private UserService userService;  
  
    @Test  
    void testGetUserById() {  
        User user = new User();  
        user.setId(2L);  
        user.setName("Anitha");  
  
        when(userRepository.findById(2L)).thenReturn(Optional.of(user));  
  
        User result = userService.getUserById(2L);  
  
        assertEquals("Anitha", result.getName());  
    }  
}
```

Exercise 3: Mocking a Service Dependency in an Integration Test

```
import org.junit.jupiter.api.Test;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;  
import org.springframework.boot.test.context.SpringBootTest;  
import org.springframework.boot.test.mock.mockito.MockBean;  
import org.springframework.test.web.servlet.MockMvc;  
  
import static org.mockito.Mockito.*;  
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;  
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;  
  
@SpringBootTest  
@AutoConfigureMockMvc  
public class UserIntegrationMockTest {  
  
    @Autowired  
    private MockMvc mockMvc;  
  
    @MockBean  
    private UserService userService;  
  
    @Test  
    void testGetUserIntegration() throws Exception {  
        User user = new User();  
        user.setId(1L);  
        user.setName("Divya");  
  
        when(userService.getUserById(1L)).thenReturn(user);  
  
        mockMvc.perform(get("/users/1"))  
            .andExpect(status().isOk())  
            .andExpect(jsonPath("$.name").value("Divya"));  
    }  
}
```

```
}  
}
```

7. Logging using SLF4J

pom.xml dependencies used for all 3 exercises:

```
<dependency>  
  <groupId>org.slf4j</groupId>  
  <artifactId>slf4j-api</artifactId>  
  <version>1.7.30</version>  
</dependency>  
<dependency>  
  <groupId>ch.qos.logback</groupId>  
  <artifactId>logback-classic</artifactId>  
  <version>1.2.3</version>  
</dependency>
```

Exercise 1: Logging Error Messages and Warning Levels

```
import org.slf4j.Logger;  
import org.slf4j.LoggerFactory;  
  
public class LoggingExample {  
  
    private static final Logger logger = LoggerFactory.getLogger(LoggingExample.class);  
  
    public static void main(String[] args) {  
        logger.error("This is an error message");  
        logger.warn("This is a warning message");  
    }  
}
```

Exercise 2: Parameterized Logging

```
import org.slf4j.Logger;  
import org.slf4j.LoggerFactory;  
  
public class ParameterizedLoggingExample {  
  
    private static final Logger logger = LoggerFactory.getLogger(ParameterizedLoggingExample.class);  
  
    public static void main(String[] args) {  
        String userName = "Sneha";  
        int userId = 101;  
  
        logger.info("User {} with id {} logged in", userName, userId);  
        logger.debug("Processing request for user: {}", userName);  
    }  
}
```

Exercise 3: Using Different Appenders

logback.xml placed in src/main/resources:

```
<configuration>  
  <appender name="console" class="ch.qos.logback.core.ConsoleAppender">  
    <encoder>  
      <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>  
    </encoder>  
  </appender>
```



```

<appender name="file" class="ch.qos.logback.core.FileAppender">
  <file>app.log</file>
  <encoder>
    <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
  </encoder>
</appender>

<root level="debug">
  <appender-ref ref="console" />
  <appender-ref ref="file" />
</root>
</configuration>

```

```

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class AppenderExample {

    private static final Logger logger = LoggerFactory.getLogger(AppenderExample.class);

    public static void main(String[] args) {
        logger.debug("This goes to both console and file appender");
        logger.info("Application started successfully");
        logger.error("This is an error message logged to both appenders");
    }
}

```